



PROMPT TEMPLATES

Quick Learning Goal

In this module, we learn how to create reusable prompts whose values can change at runtime. We start with normal prompts, move to PromptTemplate, then to ChatPromptTemplate, and finally build practical AI applications.

User Input



Prompt Template



Formatted Prompt / Messages



Chat Model



AI Message



response.content

Module 5 Roadmap

Part	Main Topic	What Students Learn
1	Prompt Basics	Prompt, static prompt, dynamic prompt, hardcoding problem
2	PromptTemplate	Single variable, multiple variables, invoke(), input_variables
3	ChatPromptTemplate	System/Human/AI roles and dynamic chat messages
4	Applications	Requirement analysis, prompt design and complete mini projects

1. What Is a Prompt?

A prompt is the input or instruction that we give to an AI model to get a response.

- Question: What is Python?
- Instruction: Explain Python in simple English.
- Task: Write a poem about nature.
- Context: Summarize the following text.
- Role: Act as a Python teacher and explain decorators.

Remember: Prompt is not only a question. A prompt can contain a question, instruction, task, context, role and examples.

Why Is a Prompt Important?

A clear prompt gives the model a clearer requirement. A vague prompt can give a general answer; a specific prompt can give a more useful answer.



Clear Prompt



Clearer Instruction



More Useful Response

Example: "Explain Java" is general. "Explain Java to a beginner in 5 simple points with one real-life example" is much clearer.

2. Static Prompt vs Dynamic Prompt

Static Prompt

A static prompt is fixed. Its content does not change unless we change the code.

```
prompt = "Explain Python in simple English."  
response = llm.invoke(prompt)  
print(response.content)
```

Dynamic Prompt

A dynamic prompt contains one or more values that can change.

```
template = "Explain {topic} in simple English."
```

topic can be Python, Java, AWS, Docker, LangChain, etc.

Feature	Static Prompt	Dynamic Prompt
Value	Fixed	Changes at runtime
Example	Explain Python.	Explain {topic}.
Reusability	Low	High

Why Not Hardcode Everything?

If the topic changes every time, writing a new hardcoded prompt again and again is unnecessary. We need one structure and different data.

Template

+

Data

=

Final Prompt



3. PromptTemplate

PromptTemplate helps us create reusable dynamic text prompts. The reusable structure stays the same; only the values change.

Classroom Program 1 - Create a Simple PromptTemplate

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
    "Explain {topic} in simple English."
)

print(prompt_template)
print(prompt_template.input_variables)
```

Remember: The placeholder {topic} is an input variable. template.input_variables tells us which values are required.

PromptTemplate.invoke()

template.invoke(...) fills the variables and creates the final prompt value. It does not call the AI model.

Classroom Program 2 - Invoke the Template and Inspect the Result

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
    "Explain {topic} in simple English."
)
print(prompt_template.input_variables)
result = prompt_template.invoke(
    {
        "topic": "Python"
    }
)
print(type(result))
print(result)
print(result.text)
print(result.to_string())

['topic']
<class 'langchain_core.prompt_values.StringPromptValue'>
text='Explain Python in simple English.'
Explain Python in simple English.
Explain Python in simple English.
```

Remember: In this classroom example, PromptTemplate.invoke() returns a StringPromptValue. We can inspect result.text or result.to_string().



Take the Value from the User

Classroom Program 3 - User Input

```
# Take topic from the user
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
    "Explain {topic} in simple English."
)
topic = input("Enter the topic:")
result = prompt_template.invoke(
    {
        "topic":topic
    }
)
print(result.text)
```

User Input

↓
template.invoke()

↓
StringPromptValue

PromptTemplate.invoke() vs llm.invoke()

These two invoke() calls do different jobs:

Call	Purpose	Result
template.invoke(data)	Fill variables and format the prompt	Prompt value
llm.invoke(prompt)	Send the prompt to the model	AIMessage

Classroom Program 4 - PromptTemplate + ChatOpenAI

```
# Prompt Template + ChatOpenAI
import os
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") #function
llm = ChatOpenAI(model="gpt-4o",api_key=OPENAI_API_KEY)

prompt_template = PromptTemplate.from_template(
    "Explain {topic} in very simple English with one example"
)
topic = input("Enter the topic:")
result = prompt_template.invoke(
    {
        "topic":topic
    }
)
```



```
response = llm.invoke(result)
print(response.content)
```

4. Single-Variable Practical Programs

Classroom Program 5 - Recipe Assistant

```
# Recipe Assistant
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    "Give me a simple recipe for {dish}. "
    "Explain it step by step."
)

dish = input("Enter dish name: ")

prompt = template.invoke(
    {
        "dish": dish
    }
)

response = llm.invoke(prompt)

print(response.content)
```

Possible inputs discussed in class included Chicken Biryani, Vegetable Biryani, Paneer Butter Masala, Masala Dosa, Idli Sambar, Pasta, Pizza, Noodles and Sandwich.



Classroom Program 6 - Topic Explainer

```
# Topic Explainer
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    """
    Explain {topic} to a beginner.

    Use:
    - Very simple English
    - 5 important points
    - One real-life example
    """
)

topic = input("Enter topic: ")

prompt = template.invoke(
    {
        "topic": topic
    }
)

response = llm.invoke(prompt)

print(response.content)

Topic: Vector Databases
```

Remember: One sample topic discussed in class was Vector Databases.



Classroom Program 7 - Short Note About Technology Generator

Short Note about Technology Generator:

```
-----  
from langchain_core.prompts import PromptTemplate  
  
template = PromptTemplate.from_template(  
    "Write a short note about {technology}."  
)  
  
technology = input("Enter technology: ")  
  
prompt = template.invoke(  
    {  
        "technology": technology  
    }  
)  
  
print(prompt.to_string())  
Try:  
Python  
Java  
AWS  
Docker
```

Try the same template with Python, Java, AWS and Docker.

Motivational Message Generator - Template Idea

Motivational Message Generator:

Template:

Write a short motivational message about {goal}.

User may enter:

Getting a job

Learning Python

Starting a business

Improving communication

The same template should work for all.

Remember: The same template can work for different goals such as getting a job, learning Python, starting a business or improving communication.



5. PromptTemplate with Multiple Variables

A prompt template can contain more than one placeholder. Each placeholder can receive a different value.

Classroom Program 8 - Multiple Variables

A prompt template can contain more than one placeholder. Each placeholder can receive a different value.

Example:

Explain {topic} in {language}.

Explain LangChain in Telugu.

Explain Python in English.

5. Real-Life Analogy

Hello {name},

You are invited to {event}

on {date}

at {place}.

```
from langchain_core.prompts import PromptTemplate
```

```
template = PromptTemplate.from_template(  
    "Explain {topic} in {language}."  
)
```

```
prompt = template.invoke(  
    {  
        "topic": "Python",  
        "language": "English"  
    }  
)
```

```
print(prompt.to_string())  
print(template.input_variables)
```

Remember: Variable names inside the dictionary must match the placeholders in the template.

Real-Life Analogy

Hello {name},

You are invited to {event}

on {date}

at {place}.

The structure stays the same. name, event, date and place can change.



Multiple-Variable Formula

One Template

+

Different Input Values

↓

Different Formatted Prompts

+

Same Model

↓

Different AI Responses

Classroom Program 9 - Smart Learning Assistant using PromptTemplate

49. Day 2 Master Application – Smart Learning Assistant

Now combine today's concepts into one useful program.

import os

from langchain_openai import ChatOpenAI

from langchain_core.prompts import PromptTemplate

Create model

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

Create reusable template

```
template = PromptTemplate.from_template(
    """
    Teach me {topic}.

    Student Level: {level}
    Language: {language}
    Explanation Style: {style}

    Requirements:
    1. Explain in easy language.
    2. Give 5 important points.
    3. Give 2 real-life examples.
    4. Mention one common mistake.
    5. End with a one-line conclusion.
    """
)
```

Take user inputs

```
topic = input("Enter topic: ")
level = input("Enter your level: ")
language = input("Enter language: ")
style = input("Enter explanation style: ")
```



```
# Generate prompt
prompt = template.invoke(
    {
        "topic": topic,
        "level": level,
        "language": language,
        "style": style
    }
)

# Inspect generated prompt
print("\n===== GENERATED PROMPT =====")
print(prompt.to_string())

# Send to model
response = llm.invoke(prompt)

# Display response
print("\n===== AI RESPONSE =====")
print(response.content)
```

Classroom Program 10 - Travel Guide using PromptTemplate

```
24. Travel Guide Program
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    """
    Create a simple travel plan for {place}.

    Duration: {days} days
    Budget: {budget}

    Include:
    - Places to visit
    - Suggested daily plan
    - Food suggestions
    - Simple travel tips
    """
)
```



```
place = input("Enter destination: ")
days = input("Enter number of days: ")
budget = input("Enter budget type (low/medium/high): ")

prompt = template.invoke(
    {
        "place": place,
        "days": days,
        "budget": budget
    }
)

response = llm.invoke(prompt)

print("\nTravel Plan:")
print(response.content)
Possible input:
Destination: Goa
Days: 3
Budget: Medium
One program can create many travel plans.
```

Classroom Program 11 - Interview Helper using PromptTemplate

```
26. Interview Helper Program
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    """
    Generate {count} interview questions on {technology}
    for a {level} candidate.

    For every question:
    - Give the question
    - Give a simple answer
    """
)

technology = input("Enter technology: ")
level = input("Enter level: ")
count = input("How many questions?: ")
```



```
prompt = template.invoke(  
    {  
        "technology": technology,  
        "level": level,  
        "count": count  
    }  
)  
  
response = llm.invoke(prompt)  
  
print("\nInterview Questions:")  
print(response.content)  
Example:  
Technology: Python  
Level: Fresher  
Count: 5  
Excellent real-world use case.
```

Classroom Program 12 - Restaurant Name Generator using PromptTemplate

```
31. Restaurant Name Generator Program  
import os  
  
from langchain_openai import ChatOpenAI  
from langchain_core.prompts import PromptTemplate  
  
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)  
  
template = PromptTemplate.from_template(  
    """  
    Suggest 10 restaurant names.  
  
    Cuisine: {cuisine}  
    Style: {style}  
    Location: {location}  
  
    Names should be:  
    - Easy to remember  
    - Attractive  
    - Suitable for the restaurant  
    """  
)  
  
cuisine = input("Enter cuisine: ")  
style = input("Enter restaurant style: ")  
location = input("Enter location: ")
```



```
prompt = template.invoke(  
    {  
        "cuisine": cuisine,  
        "style": style,  
        "location": location  
    }  
)  
  
response = llm.invoke(prompt)  
  
print("\nRestaurant Name Ideas:")  
print(response.content)
```

6. Default Values and partial_variables

Two approaches were discussed in the classroom.

Approach 1 - Default Values at input() Level

1. Default values at input() level – It is the simplest way

```
cuisine = input("Enter cuisine [Indian]: ") or "Indian"  
style = input("Enter restaurant style [Modern]: ") or "Modern"  
location = input("Enter location [Hyderabad]: ") or "Hyderabad"
```

Cuisine	Style	Location
Indian	Modern	Hyderabad
Italian	Traditional	Mumbai
Chinese	Casual	Delhi
Mexican	Modern	Bangalore

If the user presses Enter without typing a value, the value after or is used.

Approach 2 - partial_variables

partial_variables can pre-fill some PromptTemplate variables in advance.

Classroom Program 13 - partial_variables

2. LangChain PromptTemplate can also have default-like values:
By using partial_variables we can provide default values.

partial_variables is a LangChain PromptTemplate feature that allows us to pre-fill some prompt variables with values in advance.

```
template = PromptTemplate.from_template(  
    """  
    Suggest 10 restaurant names.  
  
    Cuisine: {cuisine}
```



```
Style: {style}
Location: {location}

Names should be:
- Easy to remember
- Attractive
- Suitable for the restaurant
""",
partial_variables={
    "style": "Modern",
    "location": "Hyderabad"
}
)

prompt = template.invoke(
    {
        "cuisine": "Indian"
    }
)

cuisine = Indian      # supplied at invoke time
style = Modern        # partial variable
location = Hyderabad  # partial variable

prompt = template.invoke({
    "cuisine": "Italian",
    "style": "Luxury",
    "location": "Mumbai"
})
```

Then the explicitly supplied values override the partial values:

```
Cuisine: Italian
Style: Luxury
Location: Mumbai
```

This makes `partial_variables` useful when you have common values most of the time, but occasionally want to change them.

Remember: In the classroom notes, explicitly supplied invoke-time values were shown as overriding the pre-filled partial values.



7. Normal Python String / f-string vs PromptTemplate

PromptTemplate is useful, but it is not compulsory for every tiny program.

Classroom Program 14 - Without PromptTemplate and with f-string

without prompt template:

Example 1: Without PromptTemplate

```
topic = input("Enter topic: ")
language = input("Enter language: ")
level = input("Enter level: ")
```

```
prompt = (
    "Explain " + topic +
    " in " + language +
    " for a " + level +
    " learner."
)
```

```
print("\nGenerated Prompt:")
print(prompt)
```

Sample input:

```
Enter topic: Generative AI
Enter language: English
Enter level: beginner
```

Output:

Explain Generative AI in English for a beginner learner.



Example 2: Using f-string

```
topic = input("Enter topic: ")
language = input("Enter language: ")
level = input("Enter level: ")
```

```
prompt = f"Explain {topic} in {language} for a {level} learner."
```

```
print("\nGenerated Prompt:")
print(prompt)
```

This is shorter and cleaner.

Note:

For a small Python program, normal strings or f-strings are enough. PromptTemplate becomes useful when prompts need to be reusable, structured, and connected with larger LangChain workflows.

Small f-string Example

f-strings (Formatted String Literals) were introduced in Python 3.6.

Example

```
name = "Ravi"
age = 25
```

```
print(f"My name is {name} and I am {age} years old.")
```

Approach	Good For	Main Idea
Normal string / concatenation	Very small Python logic	Build a string manually
f-string	Small dynamic strings	Insert Python values directly
PromptTemplate	Reusable LLM prompt structure	Separate template structure from runtime data



8. ChatPromptTemplate

ChatPromptTemplate is used to create reusable and dynamic prompts made up of chat messages.

PromptTemplate



Dynamic Text

ChatPromptTemplate



Dynamic Chat Messages

This connects directly with the SystemMessage, HumanMessage and AIMessage concepts learned earlier.

Hardcoded Chat Messages - Starting Point

Classroom Program 15 - Normal Message List

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
import os
messages = [
    SystemMessage("You are a helpful Python teacher."),
    HumanMessage("Explain loops."),
    AIMessage("Loops is python's important concept"),
    HumanMessage("Explain Conditional Statements.")
]

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
response = llm.invoke(messages)
print(response.content)
```

Remember: The message list works, but the contents are hardcoded.

Create ChatPromptTemplate

Classroom Program 16 - ChatPromptTemplate.from_messages()

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage

template = ChatPromptTemplate.from_messages(
    [
        SystemMessage("You are a helpful Python teacher."),
        HumanMessage("Explain loops.")
    ]
)
```



```
or

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful Python teacher."
        ),
        (
            "human",
            "Explain loops."
        )
    ]
)
```

```
prompt = template.invoke({})
print(type(prompt))
print(prompt)
for message in prompt.messages:
    print(type(message).__name__)
    print(message.content)
    print(".....")
import os

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
response = llm.invoke(prompt)
print(response.content)
```

Remember: The classroom showed two forms: message objects and role-content tuples.

ChatPromptTemplate.input_variables

```
print(template.input_variables)
```

This helps us see which dynamic values the chat template expects.



Static System Message + Dynamic Human Message

Classroom Program 17 - Dynamic Human Message

```
static systemmessage+ dynamic HumanMessage
-----
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful Python teacher. "
            "Explain everything in very simple English."
        ),
        (
            "human",
            "Explain {topic} with one example."
        )
    ]
)

topic = input("Enter topic: ")

prompt = template.invoke(
    {
        "topic": topic
    }
)

response = llm.invoke(prompt)

print(response.content)
```



Dynamic System Message + Dynamic Human Message

Classroom Program 18 - Dynamic System and Human Messages

```
Dynamic SystemMessage+ dynamic HumanMessage
-----
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a {subject} teacher."
        ),
        (
            "human",
            "Explain {topic} in simple English."
        )
    ]
)

prompt = template.invoke(
    {
        "subject": "Python",
        "topic": "decorators"
    }
)

for message in prompt.messages:
    print(
        type(message).__name__,
        ":",
        message.content
    )
```

Complete Dynamic Teaching Assistant

Classroom Program 19 - Dynamic Teaching Assistant

```
=====
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# Create chat model
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```



```
# Create dynamic chat prompt
template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            """
            You are an expert {subject} teacher.

            Student Level: {level}
            Language: {language}

            Your job is to teach concepts clearly
            and use easy examples.
            """,
        ),
        (
            "human",
            """
            Teach me {topic}.

            Requirements:
            1. Give a simple definition.
            2. Explain why it is required.
            3. Give 5 important points.
            4. Give 2 real-life examples.
            5. Mention common mistakes.
            6. End with a one-line conclusion.
            """,
        )
    ]
)

# Get values from student
subject = input("Enter subject: ")
topic = input("Enter topic: ")
level = input("Enter student level: ")
language = input("Enter language: ")

# Format chat prompt
prompt = template.invoke(
    {
        "subject": subject,
        "topic": topic,
        "level": level,
        "language": language
    }
)

# Display generated messages
print("\n===== GENERATED MESSAGES =====")

for message in prompt.messages:
```



```
print("\nMessage Type:")
print(type(message).__name__)

print("Content:")
print(message.content)

# Call model
response = llm.invoke(prompt)

# Display final response
print("\n===== AI RESPONSE =====")
print(response.content)
```

Remember: A useful debugging habit is to print `prompt.messages` before calling the model. This shows exactly what `SystemMessage` and `HumanMessage` were generated.

Can the Same Variable Appear More Than Once?

Classroom Example - Reusing the Same Variable

Q.Can Same Variable Appear More Than Once?
Yes. Even we can reuse the same variable.

Example:

```
template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are an expert in {topic}."
        ),
        (
            "human",
            "Explain {topic} to me."
        )
    ]
)
```

Here:
{topic}
appears twice.
But we provide:

```
{
    "topic": "Python"
}
```

once.
Final:
System:
You are an expert in Python.

Human:
Explain Python to me.



Yes. A variable can appear in more than one message. We still provide its value once in the invoke dictionary.

PromptTemplate vs ChatPromptTemplate

Feature	PromptTemplate	ChatPromptTemplate
Main purpose	Dynamic text prompt	Dynamic chat messages
Roles	Not the main idea	System / Human / AI roles
Variables	Yes	Yes
Multiple variables	Yes	Yes
Best fit	Text-style prompt building	Chat-model conversations

Quick Oral Questions

Q1. What is ChatPromptTemplate? A reusable template for creating dynamic chat messages.

Q2. Where do we import it from? from langchain_core.prompts import ChatPromptTemplate

Q3. Which method creates it from message definitions?

ChatPromptTemplate.from_messages()

Q4. Can system messages contain variables? Yes.

Q5. Can human messages contain variables? Yes.

Q6. Can AI messages be included? Yes.

Q7. Does template.invoke() generate an AI answer? No. It formats the chat prompt.

Q8. Which call actually invokes the model? llm.invoke(...)



9. Application Development Approach

The practical classroom approach was requirement-first, not code-first.

Requirement Analysis



Identify Input Variables



Design Prompt



Create Template Object



Get User Input



Invoke Template



Invoke LLM



AI Message



Display Response

Remember: First ask what the application should do, what values change, what stays fixed, and what prompt should be sent. Then write Python code.

10. Complete Classroom Applications

Application 1 - Restaurant Name Generator - Version 1 using PromptTemplate

Application 1 – Restaurant Name Generator

Version 1 – Using PromptTemplate:

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = PromptTemplate.from_template(
    """
    Suggest 10 restaurant names.

    Cuisine: {cuisine}
    Location: {location}
    Style: {style}
    """
)
```



```
The names should be:
- Attractive
- Easy to remember
- Suitable for the cuisine
"""
)

cuisine = input("Enter cuisine: ")
location = input("Enter location: ")
style = input("Enter restaurant style: ")

prompt = template.invoke(
    {
        "cuisine": cuisine,
        "location": location,
        "style": style
    }
)

response = llm.invoke(prompt)

print("\nRestaurant Name Suggestions:")
print(response.content)
```

Application 1 - Restaurant Name Generator - Version 2 using ChatPromptTemplate

Version 2 – Using ChatPromptTemplate

Now let's implement the same application using chat messages.
import os

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a creative branding expert "
            "who creates restaurant names."
        ),
        (
            "human",
            """
            Suggest 10 restaurant names.

            Cuisine: {cuisine}
            """
        )
    ]
)
```



```
Location: {location}
Style: {style}

The names should be attractive
and easy to remember.
"""

    )
]
)

cuisine = input("Enter cuisine: ")
location = input("Enter location: ")
style = input("Enter style: ")

prompt = template.invoke(
    {
        "cuisine": cuisine,
        "location": location,
        "style": style
    }
)

response = llm.invoke(prompt)

print(response.content)
```

Application 2 - Travel Guide using ChatPromptTemplate

```
Application 2 – Travel Guide
-----
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful travel planner. "
            "Give practical and easy-to-follow plans."
        ),
        (
            "human",
            """
            Create a {days}-day travel plan for {place}.

            Budget: {budget}
            """
        )
    ]
)
```



```
Main Interest: {interest}

Include:
1. Places to visit
2. Daily schedule
3. Food suggestions
4. Travel tips
"""

    )
]
)

place = input("Enter destination: ")
days = input("Enter number of days: ")
budget = input("Enter budget: ")
interest = input("Enter main interest: ")

prompt = template.invoke(
    {
        "place": place,
        "days": days,
        "budget": budget,
        "interest": interest
    }
)

response = llm.invoke(prompt)

print("\nTravel Plan:")
print(response.content)
```

Application 3 - Interview Helper using ChatPromptTemplate

Application-3: Interview Helper Program

```
-----
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are an experienced technical interviewer."
        ),
        (
            "human",

```



```
"""
    Generate {count} {difficulty} interview questions
    for a {level} candidate
    on {technology}.

    For every question:
    - Give the question
    - Give a simple answer
    - Give one small example
    """
)
]
)

technology = input("Enter technology: ")
level = input("Enter candidate level: ")
count = input("Enter number of questions: ")
difficulty = input("Enter difficulty: ")

prompt = template.invoke(
    {
        "technology": technology,
        "level": level,
        "count": count,
        "difficulty": difficulty
    }
)

response = llm.invoke(prompt)

print("\nInterview Preparation:")
print(response.content)
```

Application 4 - Blog Generator

Application 4 – Blog Generator

```
-----
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a professional content writer."
        ),
    ],
)
```



```
(
    "human",
    """
    Write a blog on {topic}.

    Target Audience: {audience}
    Tone: {tone}
    Length: approximately {length}

    Include:
    - Attractive title
    - Introduction
    - Main sections
    - Useful examples
    - Conclusion
    """
)
]
)

topic = input("Enter blog topic: ")
audience = input("Enter audience: ")
tone = input("Enter tone: ")
length = input("Enter approximate length: ")

prompt = template.invoke(
    {
        "topic": topic,
        "audience": audience,
        "tone": tone,
        "length": length
    }
)

response = llm.invoke(prompt)

print("\nGenerated Blog:")
print(response.content)
```

Application 5 - Smart Learning Assistant

Application 5 – Smart Learning Assistant

Smart Learning Assistant – Complete Program
import os

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```



```
template = ChatPromptTemplate.from_messages([
    (
        "system",
        """
        You are an expert {subject} teacher.

        Student Level: {level}
        Language: {language}
        Teaching Style: {style}

        Always make the explanation
        easy to understand.
        """,
    ),
    (
        "human",
        """
        Teach me {topic}.

        Give:
        1. A simple definition
        2. Why it is required
        3. Important points
        4. {examples} simple examples
        5. One common mistake
        6. One-line conclusion
        """,
    )
])

subject = input("Enter subject: ")
topic = input("Enter topic: ")
level = input("Enter student level: ")
language = input("Enter language: ")
style = input("Enter teaching style: ")
examples = input("How many examples?: ")

prompt = template.invoke(
    {
        "subject": subject,
        "topic": topic,
        "level": level,
        "language": language,
        "style": style,
        "examples": examples
    }
)

print("\n===== GENERATED MESSAGES =====")
```



```
for message in prompt.messages:
    print("\n", type(message).__name__)
    print(message.content)

response = llm.invoke(prompt)

print("\n===== AI RESPONSE =====")
print(response.content)
```

11. Classroom Mini Challenges

Mini Challenge 1 - Product Description Generator

Mini Challenge 1 – Product Description Generator

Student requirement:

Inputs:

Product

Audience

Tone

Length

System:

You are a marketing copywriter.

Human:

Write a {length} product description

for {product}

targeting {audience}

using a {tone} tone.

Students should implement this independently.

Mini Challenge 2 - Job Preparation Assistant

Mini Challenge 2 – Job Preparation Assistant

Inputs:

Role

Experience

Days

Hours Per Day

Requirement:

Create a {days}-day preparation plan

for a {role} job

for a person with {experience} experience

who can study {hours} hours per day.

Use:

ChatPromptTemplate

Mini Challenge 3 - Email Assistant

Mini Challenge 3 – Email Assistant

Inputs:



Purpose
Recipient Type
Tone
Example:
Purpose = Request leave
Recipient Type = Manager
Tone = Professional
System:
You are a professional email-writing assistant.
Human:
Write an email for {purpose}
to my {recipient}
using a {tone} tone.

Mini Challenge 4 - Fitness Plan Concept

Mini Challenge 4 – Fitness Plan Concept

For classroom programming practice only:

Goal

Days

Experience Level

Template idea:

Create a general fitness routine
for someone with goal {goal},
training {days} days per week,
at {level} level.

{goal} → weight loss, muscle gain, general fitness, improve stamina, improve flexibility

{days} → 2, 3, 4, 5, 6

{level} → beginner, intermediate, advanced

Remember: The fitness example was for classroom programming practice. The learning point is prompt templating.

12. Final Practical Exercise - AI Learning Plan Generator

Requirement: Build an AI Learning Plan Generator with Technology, Current Level, Number of Days, Hours Per Day and Goal as inputs.

Classroom Program 20 - Learning Plan Generator

Build an AI Learning Plan Generator.

Inputs:

Technology

Current Level

Number of Days

Hours Per Day

Goal

Example:

Technology = Python



```
Current Level = Beginner
Days = 30
Hours Per Day = 2
Goal = Become job-ready
-----
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            """
            You are an experienced technical trainer.
            Create practical learning plans
            that are easy to follow.
            """
        ),
        (
            "human",
            """
            Create a learning plan for {technology}.

            Current Level: {level}
            Duration: {days} days
            Available Time: {hours} hours per day
            Goal: {goal}

            Give:
            - Day-wise or week-wise plan
            - Important topics
            - Practice suggestions
            - Small projects
            - Final revision strategy
            """
        )
    ]
)

technology = input("Enter technology: ")
level = input("Enter current level: ")
days = input("Enter number of days: ")
hours = input("Enter hours per day: ")
goal = input("Enter goal: ")

prompt = template.invoke(
    {
```



```
"technology": technology,  
"level": level,  
"days": days,  
"hours": hours,  
"goal": goal  
}  
)
```

```
response = llm.invoke(prompt)
```

```
print("\nLearning Plan:")  
print(response.content)
```



13. Common Mistakes

Mistake 1 - Starting Coding Without Prompt Design

Do not begin with template = ... immediately. First identify the requirement, changing values and fixed instructions.

Mistake 2 - Making Everything Dynamic

Do not create placeholders for values that never need to change. Keep templates simple.

Mistake 3 - Variable Name Mismatch

If the template has {place}, the invoke dictionary must use "place", not "city".

Mistake 4 - Missing Input

If the template expects place, days and budget, all required values must be supplied. Use template.input_variables to inspect them.

Mistake 5 - Confusing Template Invocation and Model Invocation

template.invoke(...) formats the prompt/messages. llm.invoke(...) calls the model.

Mistake 6 - Hardcoding API Key

Avoid api_key="sk-...". Prefer os.getenv("OPENAI_API_KEY").

Mistake 7 - Assuming PromptTemplate Is Always Required

For a tiny program, a Python f-string may be enough. Use LangChain prompt abstractions when they improve reuse and structure.

Classroom Common-Mistake Notes

Common Mistake 1 – Starting Coding Without Prompt Design

Students may immediately write:

```
template = ...
```

Ayyooo!

First ask:

What is my requirement?

Then:

What changes?

Then:

What remains fixed?

Then design prompt.

Then code.

Common Mistake 2 – Making Everything Dynamic

Don't create variables just because you can.

Bad:

```
{word1}
```

```
{word2}
```

```
{instruction1}
```

```
{instruction2}
```

```
{symbol}
```

unless those really need to change.

Keep templates simple.



Common Mistake 3 – Variable Name Mismatch

Template:

Create a plan for {place}

Input:

```
{  
  "city": "Goa"  
}
```

Wrong.

Correct:

```
{  
  "place": "Goa"  
}
```

Common Mistake 4 – Missing Input

Template expects:

place

days

budget

But dictionary provides:

place

days

budget is missing.

You can inspect:

```
print(template.input_variables)
```

Common Mistake 5 – Confusing Template Invocation and Model Invocation

Again:

```
template.invoke(...)
```

means:

Create formatted prompt/messages

Whereas:

```
llm.invoke(...)
```

means:

Call the model

Very important.

Common Mistake 6 – Hardcoding API Key

Avoid:

```
api_key="sk-xxxxxxx"
```

Prefer:

```
api_key=os.getenv("OPENAI_API_KEY")
```

The prompt should not contain API secrets.

Common Mistake 7 – Assuming PromptTemplate Is Always Required

Remember:

```
topic = "Python"
```



```
prompt = f"Explain {topic}"  
is perfectly valid Python.  
For a tiny application, an f-string may be enough.  
We use LangChain prompt abstractions when they help us build reusable and structured LLM  
workflows.
```

14. Most Important Points to Remember

1. A prompt is the input or instruction given to an AI model.
2. A static prompt is fixed.
3. A dynamic prompt contains values that can change.
4. {topic} is an example of an input variable or placeholder.
5. PromptTemplate creates reusable dynamic text prompts.
6. One PromptTemplate can contain multiple variables.
7. Variable names in the invoke dictionary must match the template placeholders.
8. template.input_variables shows required variables.
9. template.invoke(...) formats the prompt; it does not call the LLM.
10. llm.invoke(...) calls the model.
11. ChatPromptTemplate creates dynamic chat messages.
12. ChatPromptTemplate can contain System, Human and AI message definitions.
13. Variables can appear in system or human message templates.
14. The same variable can be reused in more than one place.
15. response.content gives the generated answer content from the AIMessage.
16. Requirement analysis and prompt design should come before coding.

15. Module 5 Full Revision

Concept	Simple Meaning	Example / Key Syntax
Prompt	Input / instruction to model	Explain Python.
Static Prompt	Fixed prompt	prompt = "Explain Python."
Dynamic Prompt	Contains changing values	Explain {topic}.
PromptTemplate	Reusable dynamic text	PromptTemplate.from_template(...)
Input Variable	Placeholder filled later	{topic}
Multiple Variables	More than one changing value	{topic}, {language}, {level}
ChatPromptTemplate	Reusable dynamic chat messages	ChatPromptTemplate.from_messages(...)
Template Invocation	Format prompt/messages	template.invoke(data)
Model Invocation	Call the LLM	llm.invoke(prompt)
AIMessage Content	Generated answer text	response.content



Complete Module Architecture

USER



INPUT DATA



PROMPT TEMPLATE



DYNAMIC PROMPT / MESSAGES



CHAT MODEL



AI Message



response.content



USER

One-Line Module Conclusion

Prompt Templates help us separate the reusable structure of a prompt from the dynamic data supplied by the user.

ChatPromptTemplate extends the same idea to structured System, Human and AI messages.

USER DATA

+

PROMPT TEMPLATE



FORMATTED PROMPT



MODEL



AI RESPONSE



16. Ultra-Short Coding Cheat Sheet

Concept	Meaning	Key Coding Line
Import PromptTemplate	Text prompt template	from langchain_core.prompts import PromptTemplate
Create PromptTemplate	Create reusable text structure	template = PromptTemplate.from_template("Explain {topic}")
Inspect Variables	See required placeholders	template.input_variables
Format Prompt	Fill template variables	prompt = template.invoke({"topic": "Python"})
Prompt Text	Get formatted text	prompt.to_string()
Prompt Text Property	Read StringPromptValue text	prompt.text
Import ChatPromptTemplate	Chat-message template	from langchain_core.prompts import ChatPromptTemplate
Create Chat Template	Create message template	ChatPromptTemplate.from_messages([...])
System Role	High-level behaviour	("system", "You are a helpful teacher.")
Human Role	User request	("human", "Explain {topic}.")
Format Chat Prompt	Create final messages	prompt = template.invoke({"topic": topic})
Inspect Messages	See generated message list	prompt.messages
Message Type	Check System/Human/AI	type(message).__name__
Message Content	Read each message	message.content
Call LLM	Send prompt/messages	response = llm.invoke(prompt)
AI Answer	Get generated content	response.content
Environment API Key	Read key from environment	os.getenv("OPENAI_API_KEY")
Default input() Value	Use fallback when Enter pressed	value = input("Enter value [x]: ") or "x"
Partial Variables	Pre-fill common variables	partial_variables={"style": "Modern"}
f-string	Simple Python dynamic string	prompt = f"Explain {topic}"
Reuse Same Variable	One value in many places	Use {topic} multiple times; supply topic once

Three Most Important Flows

1. PromptTemplate + Data -> StringPromptValue
2. ChatPromptTemplate + Data -> ChatPromptValue / formatted messages
3. Formatted Prompt or Messages -> llm.invoke(...) -> AIMessage -> response.content

Final Memory

```
template = PromptTemplate.from_template("Explain {topic}")
prompt = template.invoke({"topic": "Python"})
print(prompt.to_string())
template = ChatPromptTemplate.from_messages([("system", "..."), ("human", "Explain {topic}")])
prompt = template.invoke({"topic": "Python"})
response = llm.invoke(prompt)
print(response.content)
```



DURGA
Software Solutions®

These student notes intentionally remove repeated classroom explanations while keeping the important concepts, syntax, classroom programs, practical applications, common mistakes and revision points from Module 5.